Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build issue and transfer erc-20 workshop credits for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Read token_transfer=500 and token_allowance=20 from mocks; multiply by10**decimals. Transfer issuer→learner, then approve spender from learner and transferFrom learner→spender. Query balances, allowance and totalSupply. Reject an overdraft from learner.

Expected assertions: Issuer999500 WSC; learner480; spender20; totalSupply1000000 WSC.

Extension prompt

Change allowance to10 and assert transferFrom20 fails before spending10. Explain smallest-unit arithmetic. Return the exact changed source lines and positive/negative tests.